

Hello World in C++

Prof. Rizzotti

1 Il nostro primo programma

Il primo programma che realizzeremo in C++ è il classico **Hello World**.

Il suo scopo è molto semplice: visualizzare sullo schermo il messaggio **Hello World**.

```
1 #include <iostream>
2
3 int main()
4 {
5     std::cout<<"Hello World";
6
7     return 0;
8 }
```

Analizziamo le diverse parti del programma per comprenderne il funzionamento.

2 La direttiva `#include`

La prima riga del nostro programma è:

```
1 #include <iostream>
```

La parola `#include` identifica una **direttiva al preprocessore**.

Il preprocessore è uno strumento che interviene prima della compilazione del programma.

Le direttive al preprocessore sono istruzioni che iniziano con il simbolo `#` e vengono elaborate prima che il compilatore analizzi il codice C++.

In particolare, la direttiva `#include` permette di includere il contenuto di un altro file all'interno del nostro programma.

2.1 Cosa sono le librerie?

Quando programiamo, non dobbiamo necessariamente scrivere tutto il codice da zero.

Possiamo infatti utilizzare codice già scritto da altri programmatori, organizzato in **librerie**.

Le librerie mettono a disposizione funzionalità che possono essere utili per realizzare i nostri programmi.

Ad esempio, potremmo aver bisogno di:

- visualizzare un messaggio sullo schermo;
- leggere un numero inserito dall'utente;
- effettuare operazioni matematiche.

Invece di implementare personalmente tutte queste funzionalità, possiamo utilizzare quelle già disponibili nelle librerie.

Nel nostro programma abbiamo scritto:

```
1 #include <iostream>
```

`iostream` è un file di intestazione della libreria standard del C++.

Mette a disposizione gli strumenti necessari per effettuare operazioni di input e output.

La direttiva `#include` inserisce il contenuto del file indicato nel nostro codice sorgente.

Possiamo immaginare che il preprocessore **incolli il contenuto del file incluso all'interno del nostro programma**.

In questo modo possiamo utilizzare funzionalità già predisposte da altri programmatori, senza doverle riscrivere.

3 La funzione main

La seconda parte del nostro programma è:

```
1 int main()  
2 {  
3     std::cout<<"Hello World";  
4  
5     return 0;  
6 }
```

Questa è la **funzione main**.

Una funzione è un blocco di codice che contiene una serie di istruzioni destinate a svolgere una determinata operazione.

3.1 Sintassi di una funzione

La sintassi generale di una funzione in C++ è:

```
1 TIPO NOME(PARAMETRI)  
2 {  
3     // Istruzioni della funzione  
4 }
```

Analizziamo i diversi elementi:

- **TIPO**: indica il tipo del valore restituito dalla funzione;
- **NOME**: identifica la funzione;
- **PARAMETRI**: rappresentano gli eventuali valori che la funzione può ricevere;
- **Parentesi graffe**: delimitano il corpo della funzione, cioè l'insieme delle istruzioni che verranno eseguite.

Nel nostro esempio:

```
1 int main()  
2 {  
3     // Istruzioni  
4 }
```

Il tipo di ritorno è `int`, il nome della funzione è `main` e le parentesi tonde sono vuote, perché non abbiamo specificato parametri.

3.2 Il punto di ingresso del programma

La funzione `main` è particolarmente importante perché rappresenta il **punto di ingresso del programma**.

Quando avviamo un normale programma C++, l'esecuzione del nostro codice comincia dalla funzione `main`.

Le istruzioni presenti al suo interno vengono eseguite nell'ordine previsto dal programma.

Ogni normale programma C++ eseguibile deve possedere una funzione `main`.

Se non la definiamo, il processo di compilazione e collegamento non riuscirà a produrre correttamente il nostro programma eseguibile.

La funzione `main` è quindi fondamentale: indica da dove deve cominciare l'esecuzione.

4 Visualizzare un messaggio con `cout`

All'interno della funzione `main` troviamo l'istruzione:

```
1 std::cout<<"Hello World";
```

Questa istruzione permette di visualizzare il messaggio `Hello World` sullo schermo.

Analizziamola nel dettaglio.

4.1 Il namespace std

In C++, i **namespace** permettono di organizzare e identificare elementi come funzioni e variabili.

Possiamo immaginare un namespace come un contenitore che raccoglie elementi utili al nostro programma.

La libreria standard del C++ utilizza il namespace **std**.

Per accedere a un elemento appartenente a un determinato namespace, utilizziamo l'operatore:

```
1 ::
```

Nel nostro esempio:

```
1 std::cout
```

stiamo indicando che vogliamo utilizzare **cout**, che appartiene al namespace **std**.

4.2 Che cos'è cout?

cout è un oggetto della libreria standard che permette di inviare informazioni all'output standard, generalmente rappresentato dal terminale.

Per visualizzare un messaggio, utilizziamo l'operatore:

```
1 <<
```

Ad esempio:

```
1 std::cout << "Ciao!";
```

Il risultato sarà:

Ciao!

Nel nostro programma abbiamo scritto:

```
1 std::cout<<"Hello World";
```

Il testo racchiuso tra virgolette viene visualizzato sullo schermo.

Il punto e virgola ; indica la fine dell'istruzione.

5 Il tipo di ritorno della funzione main

L'ultima istruzione del nostro programma è:

```
1 return 0;
```

Ricordiamo che la funzione **main** è stata definita nel seguente modo:

```
1 int main()
```

La parola `int` indica che la funzione deve restituire un numero intero.
L'istruzione `return` permette di terminare la funzione restituendo un valore.
Nel nostro caso:

```
1 return 0;
```

restituisce il numero intero 0.

Il valore restituito dalla funzione `main` viene comunicato al sistema operativo.

Per convenzione, restituire 0 significa che il programma è terminato correttamente.

Possiamo quindi interpretare `return 0;` come un messaggio al sistema operativo:

Il programma è terminato correttamente, senza segnalare errori.

Valori diversi da zero vengono generalmente utilizzati per segnalare situazioni di errore.

6 Il compilatore

Abbiamo scritto il nostro programma utilizzando il linguaggio C++.

Tuttavia, il computer non esegue direttamente il codice sorgente che abbiamo scritto.

Per ottenere un programma eseguibile abbiamo bisogno di uno strumento chiamato **compilatore**.

Il compilatore traduce il nostro codice sorgente in una forma che può essere utilizzata per produrre il programma eseguibile.

Durante questo processo vengono anche individuati numerosi errori presenti nel codice.

Ad esempio, il compilatore può segnalare:

- istruzioni scritte in modo errato;
- variabili non dichiarate;
- errori nell'utilizzo dei tipi;
- parentesi o punti e virgola mancanti.

6.1 L'ordine delle istruzioni

Per comprendere il funzionamento dei nostri primi programmi, possiamo immaginare che il compilatore legga il codice dall'alto verso il basso.

In generale, quando utilizziamo una variabile, questa deve essere già stata dichiarata.

Consideriamo il seguente esempio:

```

1 #include <iostream>
2
3 int main()
4 {
5     int numero = 10;
6
7     std::cout << numero;
8
9     return 0;
10 }

```

Questo programma è corretto.

Prima dichiariamo la variabile `numero`, assegnandole il valore 10.

Successivamente la utilizziamo per visualizzarne il contenuto.

6.2 Esempio di errore di compilazione

Consideriamo invece:

```

1 #include <iostream>
2
3 int main()
4 {
5     std::cout << numero;
6
7     int numero = 10;
8
9     return 0;
10 }

```

In questo caso stiamo tentando di utilizzare la variabile `numero` prima di averla dichiarata.

Quando il compilatore analizza l'istruzione:

```

1 std::cout << numero;

```

non trova una dichiarazione precedente della variabile `numero`.

Di conseguenza, segnala un **errore di compilazione**.

Per correggere il programma, è sufficiente dichiarare la variabile prima di utilizzarla.

6.3 Errore di compilazione ed errore di esecuzione

Non tutti gli errori vengono individuati dal compilatore.

Consideriamo questo programma:

```

1 #include <iostream>
2
3 int main()

```

```

4 {
5     int numero = 10;
6     int divisore = 0;
7
8     int risultato = numero / divisore;
9
10    std::cout << risultato;
11
12    return 0;
13 }

```

Le variabili sono state dichiarate correttamente e la sintassi delle istruzioni è valida.

Tuttavia, il programma tenta di effettuare una divisione per zero.

La divisione intera per zero produce un **comportamento non definito**.

Il compilatore potrebbe segnalare il problema, ma non è obbligato a farlo.

Se il programma viene eseguito, potrebbe interrompersi o manifestare altri comportamenti imprevedibili.

È quindi importante distinguere tra due situazioni:

- **Errore di compilazione:** il codice contiene un errore che impedisce di produrre correttamente il programma;
- **Errore durante l'esecuzione:** il programma viene compilato, ma durante il suo funzionamento si verifica un problema.

Un programma che viene compilato correttamente non è necessariamente un programma privo di errori.

¹Parti di questo documento sono output di LLM.